

Morris Traversal

Morris Traversal modifies the tree temporarily by creating threads (predecessor->right = current) so that we can traverse the tree in $O(1)$ extra space.

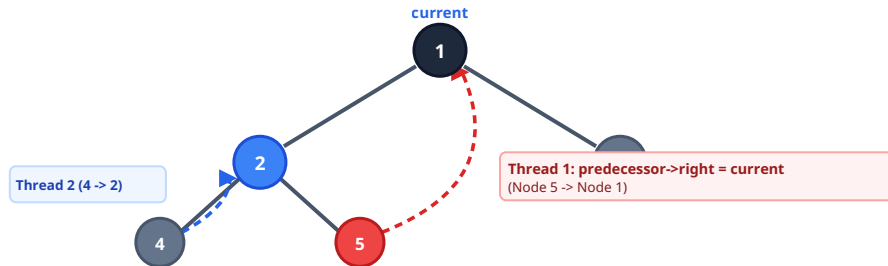
Time Complexity: $O(n)$

Space Complexity: $O(1)$

1. Morris Inorder Traversal

Left → Root → Right

SUBTREE & THREAD DIAGRAM: INORDER TRAVERSAL THREADING



Algorithm

```
current = root
while(current != NULL)
    if(current->left == NULL)
        Visit(current)
        current = current->right
    else
        predecessor = rightmost node of current->left

        while(predecessor->right != NULL &&
            predecessor->right != current)
            predecessor = predecessor->right

        if(predecessor->right == NULL)
            predecessor->right = current
            current = current->left
        else
            predecessor->right = NULL
            Visit(current)
            current = current->right
```

Code (C++)

```
vector<int> inorderTraversal(TreeNode* root) {
    vector<int> ans;
    TreeNode* cur = root;

    while (cur) {
        if (cur->left == NULL) {
            ans.push_back(cur->val);
            cur = cur->right;
        }
        else {
            TreeNode* prev = cur->left;

            while (prev->right && prev->right != cur)
                prev = prev->right;

            if (prev->right == NULL) {
                prev->right = cur;
                cur = cur->left;
            }
            else {
                prev->right = NULL;
                ans.push_back(cur->val);
                cur = cur->right;
            }
        }
    }

    return ans;
}
```

2. Morris Preorder Traversal

Root → Left → Right

Difference from Inorder

The only difference is when we visit the node.

Inorder: Visit when removing the thread. | **Preorder:** Visit before creating the thread.

Algorithm

```
current = root

while(current != NULL)

    if(current->left == NULL)

        Visit(current)
        current = current->right

    else

        predecessor = rightmost node of current->left

        while(predecessor->right != NULL &&
            predecessor->right != current)

            predecessor = predecessor->right

        if(predecessor->right == NULL)

            Visit(current)
            predecessor->right = current
            current = current->left

        else

            predecessor->right = NULL
            current = current->right
```

Code (C++)

```
vector<int> preorderTraversal(TreeNode* root) {

    vector<int> ans;
    TreeNode* cur = root;

    while (cur) {

        if (cur->left == NULL) {
            ans.push_back(cur->val);
            cur = cur->right;
        }
        else {

            TreeNode* prev = cur->left;

            while (prev->right && prev->right != cur)
                prev = prev->right;

            if (prev->right == NULL) {

                ans.push_back(cur->val);
                prev->right = cur;
                cur = cur->left;

            } else {

                prev->right = NULL;
                cur = cur->right;

            }

        }

    }

    return ans;
}
```

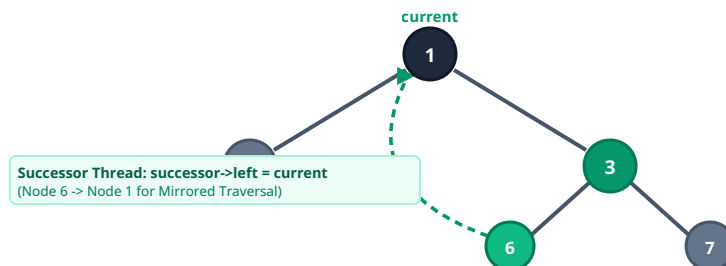
3. Morris Postorder Traversal Left → Right → Root

Traversal Trick

This is the hardest one. Unlike inorder and preorder, there is no direct Morris postorder traversal.

We use this trick: **Root → Right → Left (reverse preorder)**, then reverse the result to get **Left → Right → Root**.

SUBTREE & THREAD DIAGRAM: MIRRORED POSTORDER TRAVERSAL THREADING



Algorithm

```
current = root
while(current != NULL)
    if(current->right == NULL)
        Visit(current)
        current = current->left
    else
        successor = leftmost node of current->right
        while(successor->left != NULL &&
            successor->left != current)
            successor = successor->left
        if(successor->left == NULL)
            Visit(current)
            successor->left = current
            current = current->right
        else
            successor->left = NULL
            current = current->left
Reverse the answer.
```

Code (C++)

```
vector<int> postorderTraversal(TreeNode* root) {
    vector<int> ans;
    TreeNode* cur = root;
    while (cur) {
        if (cur->right == NULL) {
            ans.push_back(cur->val);
            cur = cur->left;
        } else {
            TreeNode* succ = cur->right;
            while (succ->left && succ->left != cur)
                succ = succ->left;
            if (succ->left == NULL) {
                ans.push_back(cur->val);
                succ->left = cur;
                cur = cur->right;
            } else {
                succ->left = NULL;
                cur = cur->left;
            }
        }
    }
    reverse(ans.begin(), ans.end());
    return ans;
}
```

One-Line Memory Trick

Inorder	Preorder	Postorder
No Left Visit Right	No Left Visit Right	No Right Visit Left
Has Left	Has Left	Has Right
Find Predecessor	Find Predecessor	Find Successor
No Thread Create Thread Left	No Thread Visit Create Thread Left	No Thread Visit Create Thread Right
Thread Exists Remove Thread Visit Right	Thread Exists Remove Thread Right	Thread Exists Remove Thread Left
		Reverse Answer

Summary Table

Traversal	Order	Thread Created At	Visit Position	Final Step
Inorder	Left → Root → Right	Rightmost node of left subtree	After removing thread	None
Preorder	Root → Left → Right	Rightmost node of left subtree	Before creating thread	None
Postorder	Left → Right → Root	Leftmost node of right subtree	Before creating thread (Root → Right → Left)	Reverse the result

The Key Pattern to Remember

- **Inorder:** Visit after coming back from the left subtree.
- **Preorder:** Visit before going to the left subtree.
- **Postorder:** Perform a mirrored preorder (Root → Right → Left) using Morris traversal, then reverse the collected result to obtain Left → Right → Root.